LiDAR Point Cloud Denoising for Autonomous Driving Applications

Technical Summary of Implementation Results

CPG No. 5 — Department of Computer Science and Engineering,
Thapar Institute of Engineering and Technology, Patiala

July 2026

1. System Implemented

The pipeline proposed in Section 5 of the project proposal has been implemented in full. Raw KITTI Velodyne scans are preprocessed (NaN/Inf removal, 60 m range gating, 0.15 m voxel-grid downsampling), corrupted with the three noise families identified in the Need Analysis (Gaussian jitter, $\sigma = 0.05$ m; uniform outliers, 8%; fog-style near-range backscatter, 4%, exponentially distributed range), and denoised by *both* approaches named in the methodology: (a) a statistical baseline chaining Statistical Outlier Removal ($k = 20$, $\alpha = 2.0$) and Radius Outlier Removal (4 neighbours within 0.6 m), and (b) two learning-based models trained on paired noisy/clean patches — a PointNet autoencoder (0.83M parameters, Chamfer loss) and a lightweight DGCNN (0.22M parameters) that regresses per-point displacements and classifies injected noise points for removal.

Data. 262 scans from two public KITTI raw drives (2011_09_26, drives 0001 and 0005), split 213/26/28 into train/validation/test by frame. The official KITTI 3D-object Velodyne archive (29 GB, login-gated) was deliberately not used: the raw split provides identical sensor data at 490 MB, sufficient for denoising experiments. Training used ∼39,000 patches of 256 points; the largest run feasible on team hardware (Apple M1, MPS backend) was 30 epochs for the autoencoder and 10 for the DGCNN.

2. Reconstruction Quality (Proposal Table-1 axes)

Averages over 10 held-out test frames; CD = Chamfer Distance, HD_{95} = 95th-percentile Hausdorff (robust to single stragglers).

The statistical baseline remains the strongest pure-reconstruction method, consistent with

Method	CD (m)	HD_{95} (m)	CD impr.	Retention	Latency
Noisy input	0.403	—	—	—	—
SOR + ROR	**0.095**	**0.136**	**75.5%**	88.6%	**104 ms**
PointNet AE	0.374	1.835	5.4%	165.6%	282 ms
DGCNN	0.162	0.156	59.1%	92.3%	3,011 ms

the literature survey's expectation that classical filters are highly effective against *additive outlier* noise. The DGCNN approaches it (and matches it on HD_{95}) while learning the noise model rather than assuming it. The autoencoder under-performs: reconstructing patches through a global bottleneck over-smooths fine scan-line structure and inflates the cloud (retention >100%), a known limitation of set-decoder architectures. An ablation over noise intensity (σ = 0.02–0.12 m) shows SOR+ROR and DGCNN degrading gracefully and in parallel, while the AE's error is dominated by reconstruction blur and is insensitive to σ. Sweeping SOR parameters confirms the proposal's tuning discussion: tighter thresholds ($\alpha = 1.0$) improve CD by ∼6% over the default on this noise profile.

3. Downstream Detection Impact

KITTI raw frames carry no ground-truth boxes, so a *detection consistency* protocol was used: a KITTI-pretrained PointPillars detector (MMDetection3D, CPU inference) is run on each clean cloud and its boxes taken as reference; recall/precision of the detections on noisy and denoised clouds are measured against that reference (BEV centre matching, 1 m tolerance, score ≥ 0.4, 10 frames).

Condition	Consistency recall	Consistency precision
Noisy input	0.760	0.853
SOR + ROR	0.733	**0.975**
PointNet AE	0.067	0.750
DGCNN	**0.788**	0.870

The synthetic noise costs the detector a quarter of its detections and introduces spurious boxes. The two viable denoisers repair different failure modes: SOR+ROR almost eliminates noise-induced *false* boxes (precision 0.85→0.98) at a small recall cost, while the DGCNN recovers the most *true* detections (recall 0.79, the best of all conditions). Notably, the ranking by Chamfer Distance does not predict this: the AE, only 7% worse than noisy on CD, is catastrophic downstream (recall 0.07). This directly substantiates the research gap identified in Section 3.5 of the proposal — reconstruction metrics alone are an incomplete proxy for perception quality.

4. Real-Time Feasibility (10 Hz target)

Per-frame latency was profiled end-to-end on the Apple M1 (batch inference, patch covering, k-d tree construction included). Dynamic int8 quantisation was applied to the learning models as the Step-5 optimisation.

Configuration	**ms/frame**	**Hz**
SOR+ROR (CPU)	174	5.7
PointNet AE (MPS GPU)	138	7.3
PointNet AE (CPU fp32)	2,856	0.35
PointNet AE (CPU int8)	2,898	0.35
DGCNN (MPS GPU)	3,317	0.30
DGCNN (CPU fp32)	20,437	0.05

Honest assessment: the 10 Hz target is not yet met. The closest configurations are the GPU-batched autoencoder (7.3 Hz) and the statistical baseline (5.7 Hz, dominated by ROR's radius queries); the DGCNN's dynamic k-NN graph construction is the principal bottleneck (0.3 Hz). Quantisation reduced the AE checkpoint 3.4× (3.34→0.98 MB) — valuable for the proposal's embedded Raspberry Pi scenario — but did not improve latency, as PyTorch's qnnpack dynamic backend accelerates only the Linear layers, which are not the bottleneck. The measured path to 10 Hz, left as future work, is (i) SOR-only filtering (interior 90 ms of the 174 ms pipeline), (ii) precomputed/voxel-hashed neighbourhoods for the DGCNN, and (iii) CUDA-class hardware, on which the batched models have substantial headroom.

5. Deliverables and Scope Notes

The repository contains the modular pipeline (`src/lidar_denoise/`), a YAML-configured driver (`scripts/run_pipeline.py`), the detection evaluation (`scripts/detection_eval.py`), an interactive before/after demo (`scripts/demo.py`), 14 unit tests, all metrics/plots under `results/`, and the corrected proposal source. The YDLIDAR X2 / Raspberry Pi live-capture component of the proposal is hardware-dependent and deferred; the pipeline is sensor-agnostic and accepts any $(N, 3)$ scan.